
Implementació de *deep learning* aplicat al processament del llenguatge natural

PID_00278335

Joaquim Moré López

Temps mínim de dedicació recomanat: 3 hores



Joaquim Moré López

L'encàrrec i la creació d'aquest recurs d'aprenentatge UOC han estat coordinats pel professor: Jordi Conesa Caralt

Primera edició: febrer 2021

© d'aquesta edició, Fundació Universitat Oberta de Catalunya (FUOC)

Av. Tibidabo, 39-43, 08035 Barcelona

Autoria: Joaquim Moré López

Producció: FUOC



Els textos i imatges publicats en aquesta obra estan subjectes –llevat que s'indiqui el contrari– a una llicència Creative Commons de tipus Reconeixement-Compartir igual (BY-SA) v.3.0. Podeu modificar l'obra, reproduir-la, distribuir-la o comunicar-la públicament sempre que en citeu l'autor i la font (Fundació per a la Universitat Oberta de Catalunya), i sempre que l'obra derivada quedi subjecta a la mateixa llicència que l'obra original. La llicència completa es pot consultar a <http://creativecommons.org/licenses/by-sa/3.0/es/legalcode.ca>

Índex

Introducció	5
1. Presentació del cas d'ús	7
2. Avantatges de l'adaptació al <i>deep learning</i>	8
3. Costos de l'adaptació	10
3.1. Conversió dels textos a dades numèriques	10
3.1.1. <i>Word embeddings</i> en mètodes de <i>deep learning</i>	10
3.1.2. Costos dels vectors <i>one-hot</i>	11
3.1.3. Costos dels vectors segons un model de llenguatge ...	12
3.2. Mètodes de <i>deep learning</i> i models de llenguatge	12
3.2.1. RNN i LSTM	13
3.2.2. Models bidireccionals	13
3.2.3. <i>Transformers</i>	14
3.2.4. Avaluació dels models	14
3.2.5. Generació de models de llenguatge amb atenció	15
3.3. Entrenament d'un model de llenguatge	22
3.3.1. Predicció d'una paraula en un model BERT	22
3.3.2. Predicció d'una paraula en un model XLNet	23
3.3.3. Predicció d'una oració en un model BERT	24
3.3.4. Predicció sobre grups de paraules i entitats anomenades	24
3.3.5. Incidència en la intel·ligència artificial	25
3.4. Requisits necessaris	25
3.4.1. Llicències i corpus font	26
3.4.2. Recursos computacionals	26
3.5. <i>Fine-tuning</i>	27
Resum	29
Glossari	30
Bibliografia	31

Introducció

Aquest mòdul presenta el *deep learning* aplicat al processament del llenguatge natural (PLN) des del punt de vista de la seva implementació pràctica. El mòdul té tres apartats.

Al primer apartat es presenta un cas d'ús de l'empresa fictícia Sense&Suitability, la qual vol adoptar els seus mètodes de PLN al *deep learning*. Al segon s'expliquen els beneficis del *deep learning* respecte als mètodes de *machine learning* tradicionals. Encara que els beneficis ja estan explicats al mòdul «Introducció al *deep learning* aplicat al processament del llenguatge natural», aquest apartat exposa de manera concisa les raons per les quals valen la pena els costos que comporta per a l'empresa l'adaptació al *deep learning*. Aquests costos s'expliquen al tercer apartat, que és la part més tècnica i més extensa i que narra el debat entre els membres de l'empresa encarregats del projecte.

1. Presentació del cas d'ús

En Peter, el coordinador del grup de lingüística computacional de l'empresa Sense&Suitability, es planteja adaptar els classificadors de textos elaborats amb mètodes de *machine learning* al paradigma del *deep learning*. Reconeix en el seu fur intern que es planteja aquesta adaptació mogut pel *boom* dels sistemes de *deep learning* i pel fet que sembla que, si no adoptes el *deep learning* com a paradigma, no estàs en sintonia.

De totes maneres, vol estar segur que l'adaptació al *deep learning* serà beneficiosa per a l'empresa, no solament perquè pot millorar els classificadors que ha generat, sinó perquè li atreu la possibilitat de transferir i incorporar mòduls de PLN en projectes ambiciosos relacionats amb la intel·ligència artificial (generadors de descripcions d'imatges, sistemes de recomanació, cercadors intel·ligents, models de raonament, sistemes de pregunta-resposta, etc.), un benefici que comporta un cost assumible, per descomptat. Per això, es reuneix amb en Joseph, l'enginyer informàtic especialista en *machine learning*, i li demana la seva opinió sobre el canvi d'orientació cap al *deep learning*.

2. Avantatges de l'adaptació al *deep learning*

En Joseph opina que l'adaptació al *deep learning* està molt justificada des del punt de vista tècnic. En comptes d'aplicar diferents algorismes de classificació (SVM, regressió logística, etc.) segons la seva idoneïtat per classificar un tipus de text, es tindria una sola xarxa neuronal per classificar grans quantitats de documents de qualsevol tipus, una xarxa neuronal amb diverses capes, l'última de les quals podria ser la capa d'entrada d'una altra xarxa neuronal especialitzada en un altre tipus de tasca vinculada a la intel·ligència artificial. A més, el programari de codi lliure per a *deep learning* (TensorFlow, Keras, PyTorch, etc.) ja està preparat per processar grans quantitats d'informació en diferents formats (text, imatges, etc.). No obstant això, prefereix que sigui la Beth, l'especialista en lingüística computacional, filòloga i traductora, qui opini sobre la seva adequació des del punt de vista lingüístic.

En Peter es reuneix amb la Beth i li pregunta com veu el canvi a *deep learning* dels seus classificadors basats en *machine learning*. La Beth sempre ha vist el *deep learning* amb un cert desdeny perquè, segons la seva manera de veure, el *deep learning* en el PLN ha fet que la lingüística computacional hagi passat a mans dels matemàtics i enginyers informàtics. Per això, reconeix positivament que en Peter tingui en compte l'opinió d'una lingüista filòloga i no es deixi portar únicament pel parer d'en Joseph.

La Beth diu a en Peter que, com que Sense&Suitability ha apostat per no codificar el coneixement lingüístic amb regles, diccionaris, etc., és bo que sàpiga que el *deep learning* permet superar algunes limitacions importants dels mètodes de *machine learning* que han aplicat. Els mètodes de *machine learning* que han utilitzat es basen en un model anomenat *bag of words* (BOW), que no recull informació contextual d'una paraula, és a dir, no té en compte les paraules que la precedeixen i la segueixen. Per tant, és difícil saber si l'ús de l'adjectiu *brutal* és positiu o negatiu en una expressió com «La sèrie *The Big Bang Theory* és brutal», i tampoc no és capaç de classificar com a positiva una opinió que s'expressa en general en termes favorables encara que s'hi esmunyi alguna crítica: per exemple, «Encara que els preus són una mica cars, el menjar és excel·lent». Amb el model BOW, no es pot reconèixer que *menjar* sigui el focus d'atenció, ja que l'ordre de les paraules no es respecta i, per tant, es desfan les dependències discursives entre el focus d'atenció i el text que li serveix de suport.

La Beth sap que hi ha xarxes neuronals dissenyades expressament per desambiguar el sentit d'una paraula segons el context i també per reconèixer les paraules d'aquest context que tenen més aportació per a la desambiguació,

xarxes neuronals que han significat un gran pas per entendre correctament una frase. Segons la Beth, si es disposa d'eines que permetin a una màquina entendre correctament una frase, malgrat les paraules ambigües, dels dobles sentits, les metàfores, la ironia o el sarcasme, les conseqüències per a la intel·ligència artificial són molt potents. Per a la intel·ligència artificial, és vital tenir un coneixement correcte de les entrades per actuar amb eficiència. I disposar d'entrades lingüístiques precises representa un grandíssim avenç. Només per això ja val la pena l'adaptació al *deep learning*.

3. Costos de l'adaptació

En Peter pregunta a en Joseph i a la Beth quin seria el cost de l'adaptació. Els dos estan d'acord a considerar que els costos se centren en els aspectes següents:

- Convertir els textos en dades numèriques.
- Entrenar un model de llenguatge.
- Obtenir i explotar els recursos necessaris.
- Generar costos de computació.

3.1. Conversió dels textos a dades numèriques

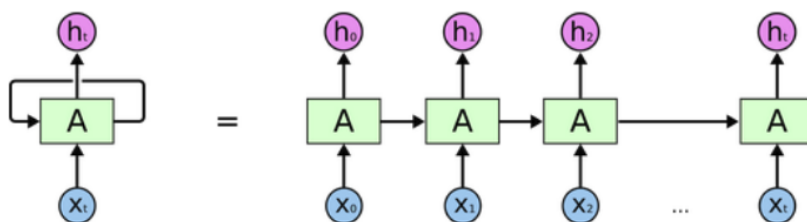
Una xarxa neuronal no admet text sinó solament dades numèriques. Quan es tracta d'un classificador (per exemple, *sentiment analysis*), la conversió de text a dada numèrica afecta l'entrada, ja que la sortida és un identificador d'aquest tipus. En la traducció automàtica, la conversió afecta tant l'entrada com la sortida, mentre que en la descripció d'imatges la conversió afecta la sortida.

3.1.1. *Word embeddings* en mètodes de *deep learning*

Les paraules del text són els elements convertits a dades numèriques en una tècnica anomenada *word embeddings*. El *word embedding* consisteix a convertir la paraula en un vector.

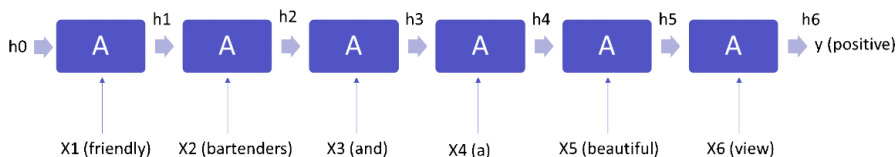
La figura 1 exemplifica l'ús de *word embeddings* en una xarxa RNN. Aquest tipus de xarxa és utilitzat per la traducció automàtica, per la descripció automàtica d'imatges i pel *sentiment analysis*.

Figura 1. *Word embeddings* en una xarxa RNN



La figura 2 representa una xarxa RNN que actua com un bucle, que està desplegat a la dreta del símbol igual. Les X són representacions vectorials de les paraules del text, això és, són els *word embeddings*. Cada A representa una cèl·lula amb uns paràmetres de pes i tendència (biaix). H_0 conté els valors inicials d'aquests paràmetres, que s'aniran actualitzant en cada unitat de temps ($h_1, h_2, \dots, h_t$). Vegem com es fa aquesta actualització en la representació d'una opinió.

Figura 2. *Word embeddings* en una RNN per al *sentiment analysis*



Font: <https://bit.ly/3dCe5PJ>.

Amb X_1 , que és el vector de la paraula *friendly*, i h_0 , es calcula h_1 ; amb X_2 , que és el vector de la paraula *bartenders*, i h_1 , es calcula h_2 , i així fins a arribar a l'últim estadi, en el qual h_6 es converteix en una sortida binària indicativa de polaritat (positiva o negativa).

Dins del bucle es manté activa la informació dels passos anteriors, i això permet que, per exemple, coneixent els estats h_0 , h_1 i h_2 es pugui predir la sortida en l'estat h_3 . Per exemple, si la seqüència fos «El rei està assegut al seu _», es podria predir que la paraula següent és *tron*.

La sortida binària és el resultat d'activar un càlcul de probabilitat basat en l'estat del node anterior. Es calcula l'error respecte al valor real de polaritat de l'opinió i després s'ajusten els paràmetres de les A amb un mètode anomenat *backpropagation through time* (BPTT) fins a arribar al mínim error possible.

3.1.2. Costos dels vectors *one-hot*

La manera més directa de convertir una paraula en una dada numèrica és convertir-la en un vector *one-hot*. El vector *one-hot* s'utilitza per distingir una paraula de les altres del vocabulari. No cal entendre el vocabulari com una llista de paraules –recordem que no podem posar dades textuais a la xarxa neuronal–, sinó com una llista d'índexs. Cada paraula del text es codifica amb un d'aquests índexs. La llista d'índexs es fa amb l'anomenat *tokenitzador*.*

* Noteu la diferència entre un tokenitzador, que en una xarxa neuronal crea una llista d'índexs, i els tokenitzadors d'analitzadors sintàctics, que creen una llista de *tokens* textuais.

Si representem el vector com una matriu $1 \times N$, N és igual al nombre de paraules no repetides del vocabulari, i cada cel·la correspon a una paraula que té un índex en aquest vocabulari. En un vocabulari de 3.000 paraules, el vector *one-hot* d'una paraula que té, suposem, l'índex 10 és una matriu 1×3.000 amb el valor 1 a la cel·la que correspon a la paraula del vocabulari amb índex 10.

Les altres cel·les tenen 0.

Processar vectors *one-hot* comporta un alt cost quan el vocabulari és molt gran: per exemple, quan el vocabulari és el de tots els articles de la Wikipedia en anglès. El cost computacional de processament de cada paraula com un vector amb un nombre N igual a la grandària d'aquest vocabulari és important. A més, a la Beth no la convencen les matrius amb cel·les plenes de zeros, excepte una per l'expressivitat gairebé nul·la del significat de les paraules que representa.

3.1.3. Costos dels vectors segons un model de llenguatge

La Beth proposa vectors més avançats del tipus Word2vec o GloVe, que representen el significat de les paraules amb un N més reduït. Ara bé, el vectoritzador ha d'aprendre a vectoritzar d'acord amb un model de llenguatge coherent i representatiu.

En Peter pregunta què s'entén per un model de llenguatge coherent i representatiu. La Beth respon que el criteri dominant actualment en l'àmbit del *deep learning* és el següent: un model de llenguatge és coherent i representatiu si, donat un context, ens permet predir una paraula correctament. Llavors en Peter comprèn que la creació de vectors comportaria crear prèviament un model de llenguatge que cal validar. I la millor manera de validar-lo seria avaluar un sistema entrenat per crear aquest model. En Peter s'inquieta perquè veu costos amb els quals no comptava.

3.2. Mètodes de *deep learning* i models de llenguatge

La Beth planteja dos problemes tradicionals del PLN com a criteris per avaluar els mètodes de *deep learning* més adequats per crear un model de llenguatge.

El primer criteri és el següent: donades les frases «S'ha produït un brutal assassinat» i «Et recomano la sèrie *The Big Bang Theory*. És brutal», un bon mètode de creació de models de llenguatge és el que genera vectors diferents per als diferents significats de *brutal* segons el context.

El segon criteri és la generació de pronoms. Segons la Beth, la generació d'un pronom que concorda amb el nom al qual fa referència és un cas de predicció correcta. Això és, generar un pronom és com predir la paraula que ve a continuació en una seqüència com «Ahir vaig visitar el meu pare i vaig menjar amb _».

3.2.1. RNN i LSTM

En Peter diu que els models de llenguatge es poden elaborar amb una xarxa RNN perquè, com s'ha explicat a l'apartat 3.1.1, una xarxa RNN permet predir la paraula *tron* donat un context com «El rei està assegut al seu _».

No obstant això, la Beth apunta que les xarxes RNN no són capaces de predir un sortida que està a llarga distància. L'estat de la cel·la sempre canvia amb cada entrada, amb la qual té una relació de dependència però respecte a la qual està molt allunyat.

A l'exemple clàssic «Em vaig educar a França. Parlo francès fluidament», l'RNN pot intuir que la paraula *francès* és un idioma per aparèixer després de la paraula *parlo*. No obstant això, si el text és «Em vaig educar a França. Encara que vaig decidir tornar a Espanya quan em vaig casar, i d'això fa ja vint anys, el francès no l'he oblidat», l'RNN no és capaç de predir *francès* com una paraula que es refereix a l'idioma del país en què es va educar. Aquest és un problema típic del *sentiment analysis*, tal com il·lustra l'opinió següent: «L'hotel va ser molt decebedor. Les habitacions eren petites, a recepció hi havia un tracte terrible, encara que el restaurant va ser correcte. Quin desastre.» L'RNN no prediu *desastre* com una paraula que es refereix a *hotel*, que és a una distància considerable.

Una alternativa possible per solucionar aquests problemes de dependència a llarga distància són les xarxes *long short term memory* (LSTM). Un model de llenguatge que s'elabora amb LSTM és ULMFit. Ara bé, com apunta la Beth, les LSTM tenen una limitació: la predicció de la paraula es fa mantenint la memòria de les paraules precedents, sense tenir en compte que les paraules posteriors també poden tenir pes per predir la paraula correcta. Per exemple, a «Un jugador de ____ professional és Rafa Nadal», la paraula *tennis* es prediu per la presència del context a l'esquerra però sobretot per la presència de *Rafa Nadal* a la dreta.

Lectura recomanada

Per saber més sobre les xarxes LSTM, recomanem llegir el capítol 11 del llibre *Deep learning. Principios y fundamentos*.

3.2.2. Models bidireccionals

Per superar aquestes limitacions, a principis de l'any 2018 va aparèixer el model ELMo (Embeddings from Language Model, de l'Institut Allen), que utilitza dues LSTM, una cap endavant i una altra cap enrere, però, i és important puntualitzar això segons la Beth, no miren en les dues direccions alhora. El model que sí que ho fa, i que és l'iniciador dels anomenats models *transformers* bidireccionals, és el BERT (Bidirectional Encoder Representations from Transformers) de Google, que va aparèixer a finals del 2018 i que ha marcat un abans i un després en el PLN, perquè implementa la tècnica basada en la noció d'atenció tal com està explicada al famós article presentat per Google «Attention is all you need» (Vaswani i altres, 2017).

Els models bidireccionals superen les limitacions dels models seqüencials ba-

sats en RNN i LSTM perquè permeten aprendre al mateix temps el significat d'una paraula segons les paraules que la precedeixen i la segueixen. Són considerats com una fita en el PLN perquè no estan limitats a controlar contextos parcials com les RNN i LSTM, sinó que poden controlar tot el context alhora. No solament modelitzen el significat de les paraules, sinó que també modelitzen el significat de les frases senceres i el coneixement del llenguatge en general.

En Peter demana a en Joseph i a la Beth que li mostrin la implementació d'un model BERT amb exemples pràctics en les tasques de predir una paraula i de vectoritzar dos sentits diferents d'una paraula. Aquests exemples estan al *notebook* «Models bidireccionals del llenguatge».

3.2.3. *Transformers*

Les xarxes que construeixen models basats en l'atenció, com BERT, són conegudes com a *transformers*. Tenen un codificador que codifica el context i un descodificador que el transforma per fer prediccions pel que fa a una tasca concreta. Les nocions de codificador i descodificador són habituals en la traducció automàtica. De fet, els *transformers* van sorgir com un mètode pensat per a la traducció automàtica i mantenen aquest nom perquè en la traducció automàtica transformen una seqüència d'entrada en una seqüència diferent en una altra llengua.

No tots els *transformers* són bidireccionals com BERT. El model GPT d'OpenAI* no ho és perquè la informació flueix d'esquerra a dreta.

* <https://bit.ly/3o1xM7B>

3.2.4. *Avaluació dels models*

Amb la irrupció dels *transformers*, l'avaluació de sistemes de PLN ha canviat. Abans, les avaluacions es dissenyaven per a tasques concretes. Ara, els models aspiren a ser representacions d'un bon coneixement del llenguatge en general. Tenint en compte aquesta ambició, un equip de la Universitat de Nova York va pensar d'avaluar un model aplicant-lo a una sèrie de tasques representatives del coneixement del llenguatge. Aquestes tasques tenen en comú el processament d'entrades textuais i la generació d'una predicció. Entre aquestes tasques hi ha la de classificar una opinió segons la seva polaritat, el reconeixement d'entitats anomenades (NER), etc. Per a cada tasca, el model té una puntuació, i la valoració final és la puntuació mitjana, anomenada GLUE (General Language Understanding Evaluation) *score*.*

* <https://bit.ly/2GuHPC5>

Fins ara els models *transformers* són els que tenen valors més alts.** D'altra banda, BERT s'ha pres com a punt de referència per a altres tasques d'avaluació, conegudes com a SuperGLUE i presentades a mitjan 2019 per Google, Facebook, la Universitat de Washington i la de Nova York.***

** <https://bit.ly/3db4a3e>

*** <https://bit.ly/2SDQ8h6>

Al moment d'escriure aquest mòdul, el model ERNIE (Enhanced Representati-

on through kNnowledge IntEgration), de Baidu, Inc., ocupa la primera posició en el rànquing de GLUE. És un model elaborat per un generador que es caracteritza per integrar informació que troba rellevant durant el seu entrenament per a les tasques en les quals s'aplica (entitats anomenades, proximitat entre oracions, etc.). A diferència de BERT, que té models preentrenats en diferents llengües, entre les quals hi ha l'espanyol, ERNIE només té models de l'anglès i el xinès.

3.2.5. Generació de models de llenguatge amb atenció

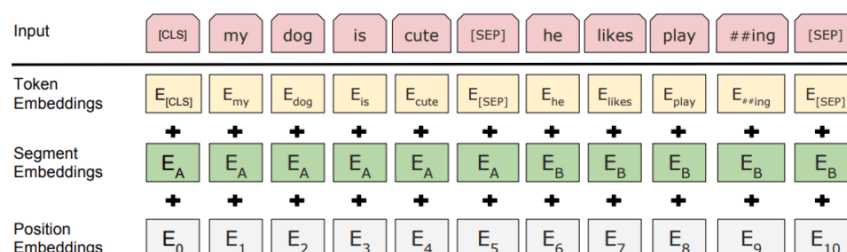
En Peter està convençut que els models basats en el mecanisme d'atenció són els més adequats. Ara bé, és conscient que es tracta d'un canvi de paradigma i vol saber els costos que aquest canvi pot comportar. En Joseph li dona les explicacions següents basant-se en el model BERT, que resumim:

1) Transformació del text. L'entrada es transforma amb l'anomenat *wordpiece tokenizer*, que pren, per reduir el vocabulari, l'arrel i els sufixos com a *tokens* separats. Així, en comptes d'afegir al vocabulari una paraula com *playing*, aquesta es transforma en dos *tokens* del vocabulari, que són *play* i *##ing*. Els dos coixinets indiquen que el *token* és un sufix. Al principi i al final de la frase s'hi afegeixen uns *tokens* especials, que són [CLS] i [SEP], respectivament. Són els anomenats *tokens* separadors perquè indiquen el principi i el final d'una frase. Veurem la importància d'aquests *tokens* més endavant.

2) Representació de l'entrada. La finalitat del codificador és codificar la frase sencera, no seqüències parcials. Per això, l'entrada es representa com la suma de tres tipus d'*embeddings* (vegeu la figura 3):

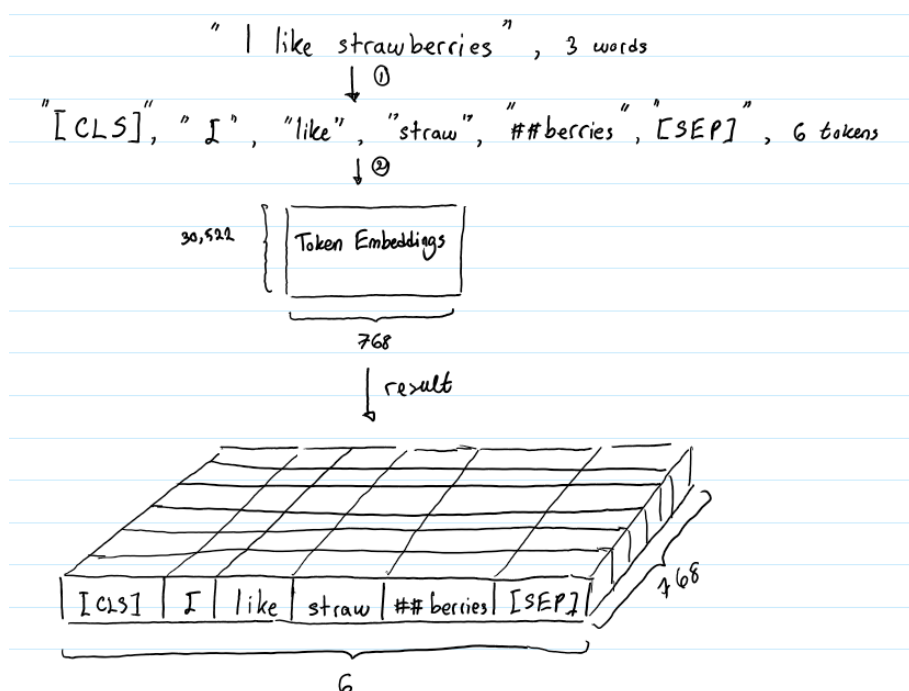
- *token embeddings*
- *segment embeddings*
- *position embeddings*

Figura 3. Representació de l'entrada amb tres tipus d'*embeddings*



Font: Devlin i altres (2019). BERT: *pre-training of deep bidirectional transformers for language understanding*.

Els *token embeddings* transformen les paraules en un vector d'una longitud fixa i preestablerta (vegeu la figura 4). Els vectors de *tokens* s'elaboren amb *one-hot vectors*, Word2vec, GloVe, etc.

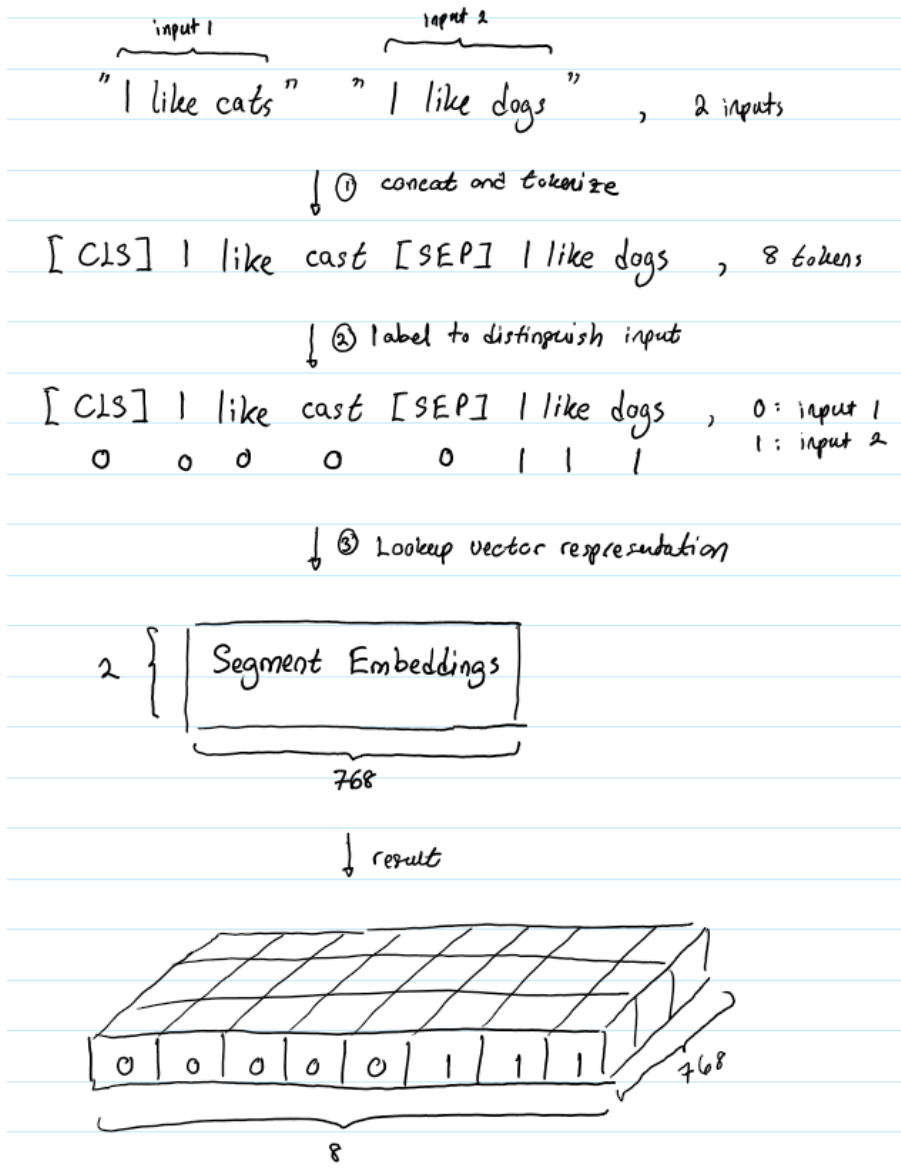
Figura 4. *Token embeddings* amb una longitud de 768

Font: <https://bit.ly/3d1r11l>.

Els *segment embeddings* generen tants vectors com segments hi ha al text. Suposem que els separadors separen dos segments que són oracions. Els *segment embeddings* generen dos vectors. En el primer s'assigna un índex 0 a tots els *tokens* que pertanyen al primer segment. En el segon s'assigna un índex 1 a tots els *tokens* que pertanyen al segon segment (vegeu la figura 5). Si l'entrada té un sol segment, aquest seria el vector si fos el primer segment.

Els *positional embeddings* són vectors que donen informació sobre la posició dels *tokens* a la frase. Són importants perquè assignen vectors diferents a paraules que són iguals. Per exemple, el *positional embedding* de la paraula *I* en una posició inicial serà diferent del *positional embedding* de la mateixa paraula en una posició intermèdia. La manera com es calculen els *position embeddings* s'explica a l'apartat 3.5 de l'article «Attention is all you need».

3) Self-attention. La codificació de l'entrada passa per la capa dedicada a donar a tota la seqüència de paraules una nova representació segons el mecanisme de *multi-head attention*. Abans d'explicar en què consisteix la *multi-head attention*, és millor presentar el mecanisme anomenat *self-attention*.

Figura 5. *Segment embeddings* amb una longitud de 768Font: <https://bit.ly/3nmqsnh>.

A l'article «Attention is all you need», el mecanisme d'*attention* es defineix així:

«An attention function can be described as mapping a query and a set of key-value pairs to an output, where the query, keys, values, and output are all vectors. The output is computed as a weighted sum of the values, where the weight assigned to each value is computed by a compatibility function of the query with the corresponding key.»

I aquesta és la definició de *self-attention* al mateix article:

«Self-attention, sometimes called intra-attention, is an attention mechanism relating different positions of a single sequence in order to compute a representation of the sequence.»

Prenguem com a exemple la frase «Try to run the programme». La *self-attention* determina en quina mesura les altres paraules de la seqüència contribueixen

a representar *run*. *Programme* és la paraula que contribueix més al fet que el significat de *run* no sigui 'córrer', sinó 'posar en funcionament un programa informàtic'.

Recordem que amb una LSTM això no era possible perquè *programme* ve després de *run* a la seqüència.

Com s'implementa la *self-attention*? Per entendre-ho, cal tenir en compte els conceptes següents de la definició de *attention*:

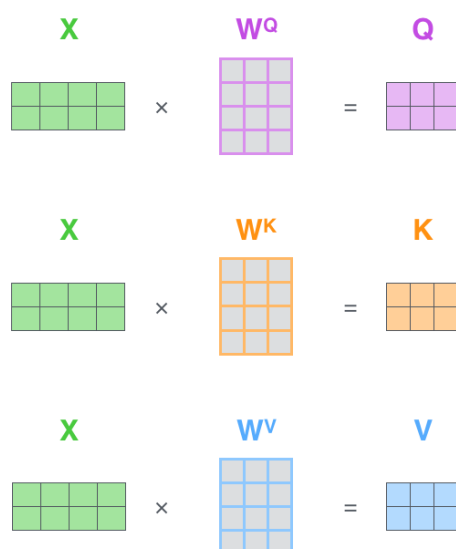
- *query*
- *key*
- *value*

Es tracta de conceptes que recorden els elements dels diccionaris de Python, però la seva funció s'entendrà si s'imagina una paraula com una persona que vol saber com els altres la veuen i la valoren per crear una versió nova de si mateixa d'acord amb aquesta visió.

Per exemple, la paraula p_1 vol saber el valor que li dona la paraula p_2 . Per tant, p_1 pregunta a p_2 amb una matriu Q (*query*). p_2 li respon amb la seva *key* (K) que té un valor (V).

La figura 6 il·lustra com es creen les matrius Q , K i V a partir de l'entrada X , que també es representa com una matriu. Cada fila de X correspon al vector d'un *token*. Després es creen tres matrius de pesos calculats aleatòriament, que són W^Q , W^K i W^V . La matriu Q és el resultat de multiplicar W per W^Q . Així mateix, K és igual a la multiplicació de W per W^K i V és igual a la multiplicació de W per W^V .

Figura 6. Creació de Q , K i V



Una vegada que tenim Q , K i V , es pot crear una representació de l'oració, que és una altra matriu on es registren els valors d'*attention* entre les paraules. El valor d'*attention* entre dos *tokens* es calcula amb la fórmula de la figura 7.

Figura 7. Fórmula del càlcul d'*attention*

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Els elements importants de la fórmula són Q , K i V . Els altres elements són paràmetres per evitar possibles biaixos en el valor obtingut. En aquesta fórmula es fa el producte escalar de Q amb la K de cadascuna de les paraules de l'oració (K^T). Així es representa la similitud entre els dos vectors. Aquesta similitud també s'anomena *attention score*. A la figura 8 es representa la matriu de tota l'oració segons l'*attention score* entre els *tokens* una vegada que s'hi ha afegit la funció *softmax*. Cada fila de la matriu representa l'*attention score* entre un *token* i els altres *tokens*.

Figura 8. Representació d'una oració segons els *attention scores*

	<i>Hello</i>	,	<i>how</i>	<i>are</i>	<i>you</i>	?	
<i>Hello</i>	$72.40 * 10^{-06}$	$1.23 * 10^{-21}$	$6.51 * 10^{-40}$	$2.62 * 10^{-22}$	$9.99 * 10^{-01}$	$4.30 * 10^{-08}$	= 1
,	$1.00 * 10^{+00}$	$7.51 * 10^{-30}$	$1.54 * 10^{-17}$	$9.91 * 10^{-13}$	$8.15 * 10^{-69}$	$1.09 * 10^{-30}$	= 1
<i>how</i>	$3.12 * 10^{-70}$	$2.51 * 10^{-51}$	$2.72 * 10^{-21}$	$8.03 * 10^{-09}$	$1.29 * 10^{-07}$	$9.99 * 10^{-01}$	= 1
<i>are</i>	$2.47 * 10^{-72}$	$5.54 * 10^{-05}$	$9.80 * 10^{-01}$	$1.98 * 10^{-02}$	$2.77 * 10^{-82}$	$2.58 * 10^{-08}$	= 1
<i>you</i>	$2.67 * 10^{-05}$	$1.21 * 10^{-09}$	$9.75 * 10^{-07}$	$3.17 * 10^{-76}$	$9.99 * 10^{-01}$	$3.64 * 10^{-28}$	= 1
?	$8.59 * 10^{-47}$	$1.05 * 10^{-35}$	$9.99 * 10^{-01}$	$2.38 * 10^{-15}$	$4.21 * 10^{-27}$	$4.07 * 10^{-06}$	= 1

Font: <https://towardsdatascience.com/understand-self-attention-in-bert-intuitively-cd480cbff30b>.

L'*attention score* representa la similitud entre dos *tokens*. Ara bé, les matrius tenen la grandària següent: (nombre de paraules de la frase, nombre de paraules de la frase). Aquestes matrius no representen la frase original perquè no s'han tingut en compte els *token embeddings*. Per això, es multiplica l'*attention score* per V , que és una matriu amb la grandària següent: (nombre de paraules de la frase, nombre de dimensions del *token embedding*). El resultat és una representació dels *tokens* que mereixen més atenció i dels que no en mereixen.

Un *token* té tantes matrius resultat de multiplicar l'*attention score* per V com *tokens* hi ha a l'oració. La nova representació del *token* és un vector les dimensions del qual són la suma dels valors de cada columna en les tres matrius.

A la figura 9 s'il·lustra com es fa la nova representació de la paraula *action* a l'oració «Action gets results». I s'hi fa el mateix amb les altres paraules.

4) Multi-head attention. La *self attention* és, de fet, una qüestió més complexa. És més fiable un valor de *self attention* si el mecanisme s'executa sobre la mateixa entrada n vegades. La manera de fer-ho és partint Q , K i V en molts *heads*, que són matrius de pesos calculats aleatòriament. D'aquesta manera, la contribució dels *tokens* per al nou *embedding* d'una paraula depèn de cada *head*, amb la qual cosa és possible veure la contribució de les paraules per

Enllaç d'interès

Recomanem la il·lustració animada del procés de *self-attention* que és a l'article «Illustrated: self-attention» <https://bit.ly/30LGmya>.

determinar el sentit d'un mot segons diferents aspectes.

Figura 9. Mecanisme d'atenció per representar una paraula

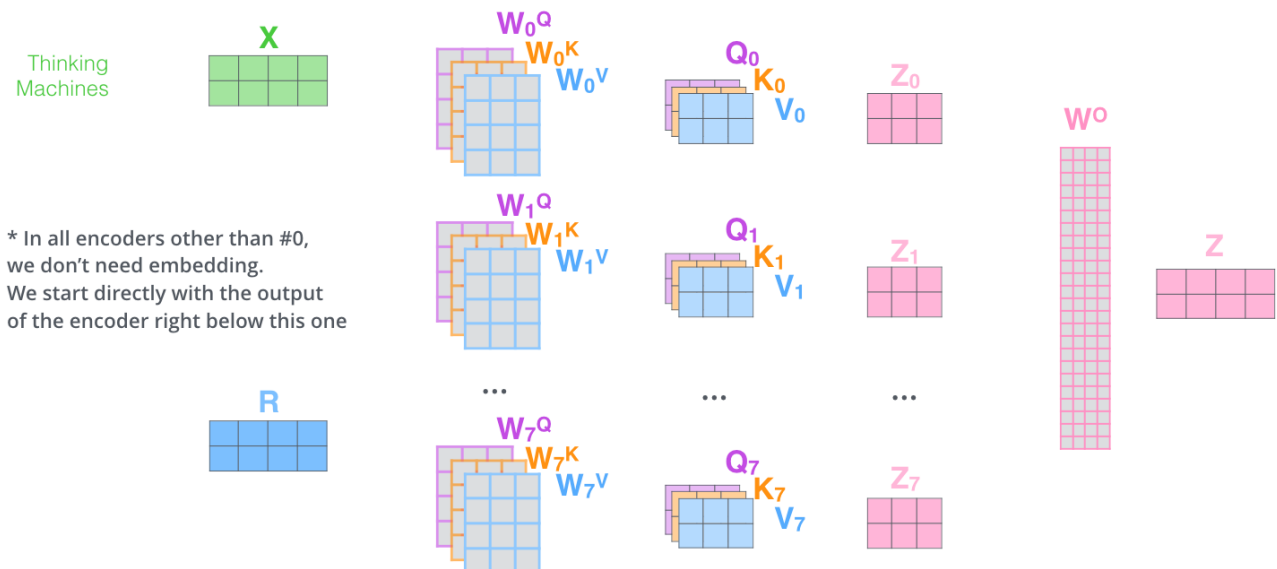
Word	q vector	k vector	v vector	score	score / 8	Softmax	Softmax * v	Sum
Action	q_1	k_1	v_1	$q_1 \cdot k_1$	$q_1 \cdot k_1 / 8$	x_{11}	$x_{11} * v_1$	z_1
gets		k_2	v_2	$q_1 \cdot k_2$	$q_1 \cdot k_2 / 8$	x_{12}	$x_{12} * v_2$	
results		k_3	v_3	$q_1 \cdot k_3$	$q_1 \cdot k_3 / 8$	x_{13}	$x_{13} * v_3$	

Font: <https://www.analyticsvidhya.com/blog/2019/06/understanding-transformers-nlp-state-of-the-art-models/>.

El resultat de calcular l'atenció per a cadascun d'aquests *heads* són N matrius per cada *head* (8, segons l'article). Per aconseguir un únic valor es concatenen i es multipliquen per una matriu calculada inicialment de manera aleatòria. Tot el procés de *multi-head attention* s'il·lustra a la figura 10.

Figura 10. Mecanisme de *multi-head attention*

- 1) This is our input sentence*
- 2) We embed each word*
- 3) Split into 8 heads. We multiply X or R with weight matrices
- 4) Calculate attention using the resulting $Q/K/V$ matrices
- 5) Concatenate the resulting Z matrices, then multiply with weight matrix W^O to produce the output of the layer



Font: <https://towardsdatascience.com/breaking-bert-down-430461f60efb>.

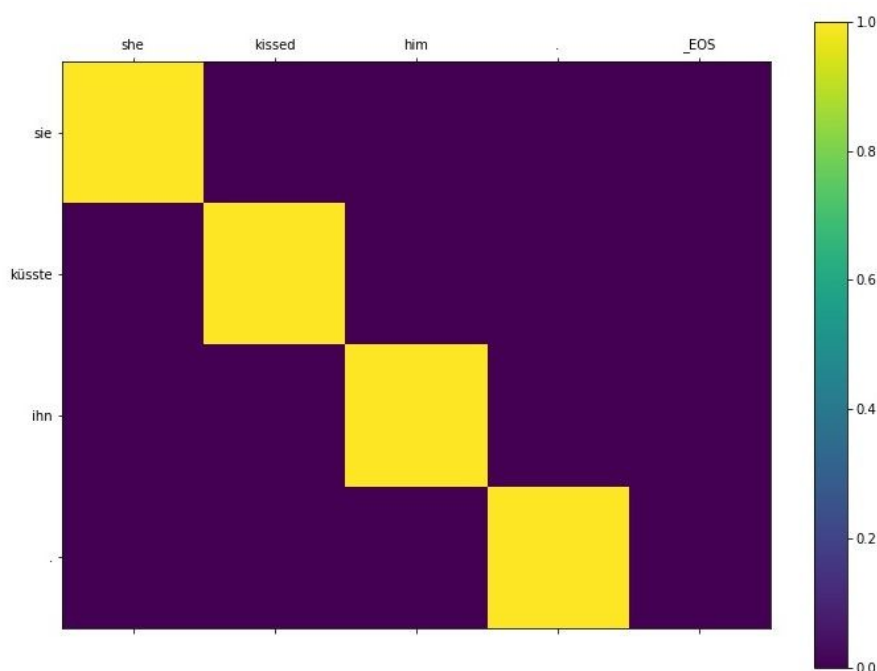
5) **Feed-forward.** Els resultats per a cada paraula es passen a una capa, que és una *feed-forward neural network*.

6) **Sortida del codificador i entrada del descodificador.** El *feed-forward* és l'últim pas del codificador. La capa, amb la sortida del codificador, es passa al descodificador perquè tingui en compte els elements de l'oració que tenen més atenció al moment de fer una predicció per a una tasca concreta.

El descodificador utilitza el mateix procediment de *multi-head attention* explicat, però l'atenció del descodificador és la *query* i l'atenció de l'*encoder* són K i V . Aquest procediment es dona en la capa anomenada *encoder-decoder attention layer*. Així s'utilitzen parts de l'entrada on es concentra la informació més rellevant.

La traducció automàtica il·lustra molt bé la interacció codificador-descodificador amb els mecanismes d'atenció. A la figura 11, en l'eix de la X hi ha les paraules de la llengua origen i en l'eix de la Y les paraules de la llengua destinació. La intensitat de les cel·les correspon als pesos de les associacions entre la paraula origen i la paraula destinació basades en l'atenció.

Figura 11. Correspondència de paraules traduïdes amb els valors d'atenció de la frase original

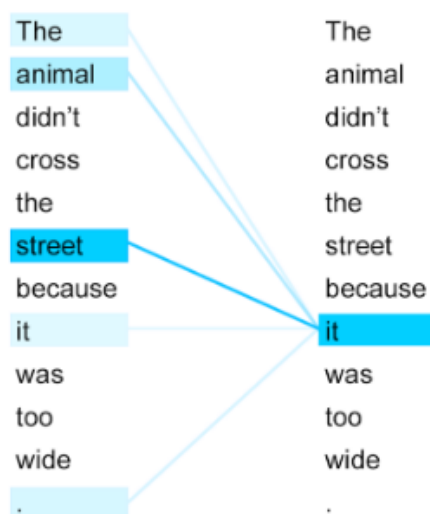


Font: <https://blog.fiorydhub.com/attention-mechanism/>.

Amb el mecanisme d'atenció, es pot predir un pronom tenint en compte la paraula amb què està més vinculada. A la figura 12 s'exemplifica la vinculació del pronom *it* amb el seu antecedent. El gruix de les línies representa els pesos de la vinculació del pronom amb les paraules de la frase.

Quan la tasca és fer un model de llenguatge que desambigui les paraules, el descodificador pren una paraula del tipus *assassinat* com un element important per predir una representació única de *brutal* en el seu sentit negatiu.

Figura 12. Vinculació del pronom amb les paraules de la frase pel seu grau d'atenció



Font: <https://www.analyticsvidhya.com/blog/2019/06/understanding-transformers-nlp-state-of-the-art-models/>.

3.3. Entrenament d'un model de llenguatge

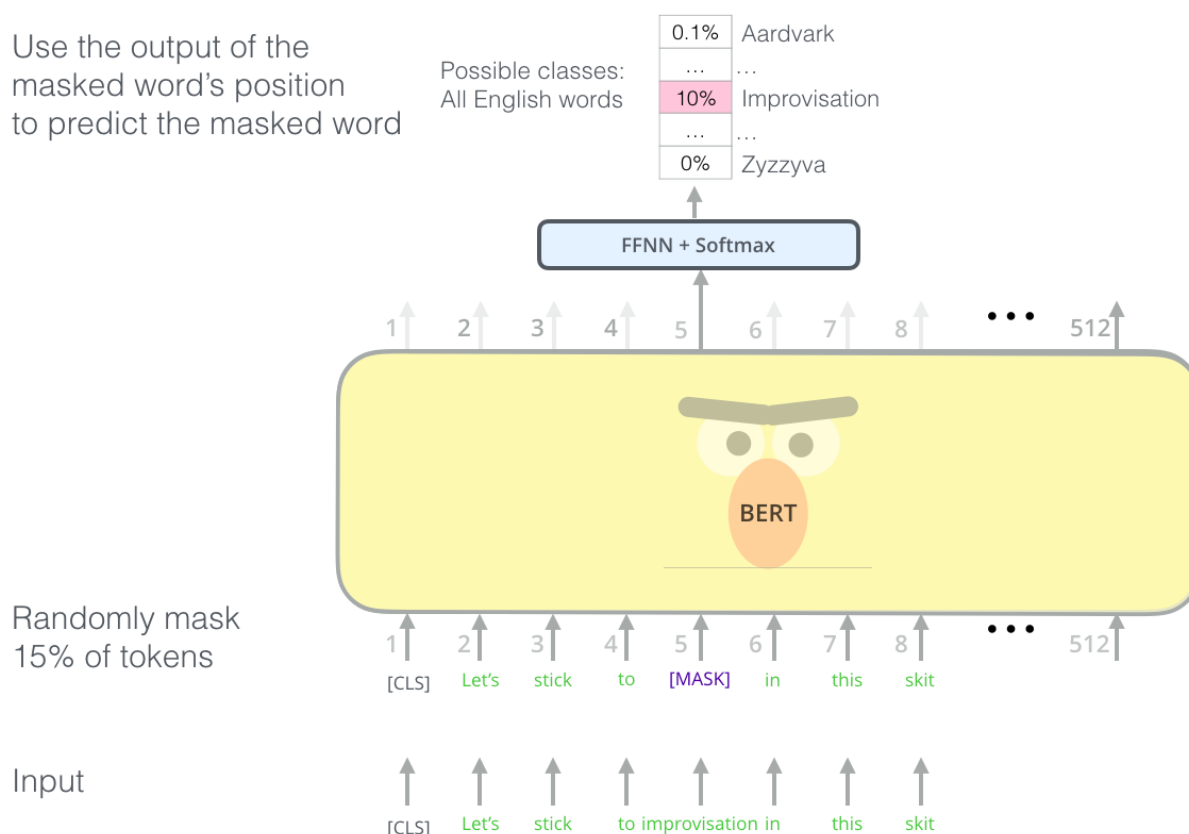
En Peter està convençut que els models bidireccionals basats en el mecanisme d'atenció són els més adequats. Però recorda que els models que aspiren a ser bidireccionals, com ELMo, no resolen aquest problema: per predir una paraula al final d'una frase, aquesta paraula hauria de mirar-se a si mateixa.

La Beth li explica la solució per entrenar un generador d'un model bidireccional com BERT. Consisteix a ocultar (*mask*) paraules de manera aleatòria –el 15 % de l'entrada– i predir aquestes paraules amb el mètode d'atenció. D'aquesta manera, ja no hi ha el problema de caure en la inconsistència que sigui necessària una paraula per predir-se a si mateixa. A més, es força el generador del model a verificar el que aprèn. La predicció es fa en una iteració i l'error de la qual anirà disminuint en iteracions posteriors.

3.3.1. Predicció d'una paraula en un model BERT

La figura 13 il·lustra el procés de predicció d'una paraula en un model BERT. La predicció es resol com un problema de classificació. Les probabilitats que la paraula emmascarada sigui una paraula del vocabulari es presenten mitjançant *softmax*.

Figura 13. Predicció d'una paraula en BERT



Font: <http://jalamar.github.io/illustrated-bert/>.

3.3.2. Predicció d'una paraula en un model XLNet

En Joseph apunta que el mètode d'emascament suggereix dubtes metodològics. Per exemple, hi ha dubtes que l'emascament no influeixi en la correcció dels models. Especialment, un punt considerat feble és que els *tokens* emascarats es prediuen en paral·lel sense tenir en compte les paraules emascarades de la frase que ja han estat predites. Vegem un exemple:

Vaig anar a [MASK] [MASK] i vaig veure el [MASK] [MASK] [MASK].

Una opció seria predir el següent:

Vaig anar a Nova York i vaig veure l'Empire State Building.

Però també podria ser:

Vaig anar a San Francisco i vaig veure l'Empire State Building.

Però aquesta opció no seria vàlida per incoherent. És a dir, que no aprèn les dependències entre les pròpies prediccions. No té memòria del que ha predit abans. Això no recorda una mica el problema que precisament les xarxes RNN i LSTM resolien?

Una proposta que pren la part bona de l'atenció i la part bona de les xarxes RNN i LSTM és XLNet, de la Universitat Carnegie Mellon. És una alternativa a BERT que s'ha presentat com un mètode que dona resultats millors. Ho fa amb una variant de model de llenguatge anomenada *permutation language*

modelling, que prediu una paraula tenint en compte el context precedent, però en lloc de predir les paraules en un ordre seqüencial, ho fa en un ordre aleatori. Cal aclarir que no es canvia l'ordre de les paraules, sinó que es canvia l'ordre en què es prediuen.

Per exemple, a l'hora de predir la frase «I like cats more than dogs», en lloc de fer-ho en l'ordre *I, like, cats, more, than, dogs* tenint en compte el context precedent, pot predir les paraules en l'ordre següent: *cats, than, I, more, dogs, like*, en el qual *than* estaria condicionat en veure *cats*, *I* estaria condicionat en veure *cats, than*, etc. Com que hi ha *positional embedding*, mentre aquest sigui coherent, els *tokens* es veuran en l'ordre correcte. D'aquesta manera, es poden veure relacions entre paraules que BERT no veu.

3.3.3. Predicció d'una oració en un model BERT

La Beth recorda a en Peter la importància que BERT no predigui solament una paraula. També s'entrena per predir una frase. En aquest cas, les entrades són dues frases, separades pel *token* SET. En l'entrenament, el 50 % del corpus té una primera frase que precedeix la segona, i en la resta apareix una frase aleatòria en la segona posició. El model ha de ser capaç de predir si la segona frase és realment la frase que ve després de la primera. Aquestes dues frases es representen en un únic *embedding*, anomenat *segment embedding*.

El mecanisme de predicció consisteix a comprovar quines paraules són més rellevants prenent SEP com a focus d'atenció. Aquest és un procediment molt astut. Mantenint el mètode d'atenció –SEP és la *query*, com el *token* d'una paraula que busca els *tokens* més vinculats a ell–, és possible determinar els *tokens* centrals de la frase, que no són molts. La predicció de la frase, per tant, es resol de la mateixa manera que la predicció de la paraula, com un problema de classificació amb la diferència que es calcula la probabilitat que una frase aleatòria estigui vinculada a la frase que la precedeix.

3.3.4. Predicció sobre grups de paraules i entitats anomenades

Aquesta predicció és la que fa ERNIE.* ERNIE s'entrena per predir grups de paraules emmascarades, entre les quals hi ha entitats anomenades. L'exemple següent explica un aspecte en què ERNIE supera BERT.

* <https://bit.ly/3nDfxGf>

Harry Potter is a series of fantasy novel written by J. K. Rowling.

BERT aprèn amb un *masked corpus* com

[MASK] Potter is a series [MASK] fantasy novel [MASK] by J. [MASK] Rowling.

BERT pot identificar *K.* per la influència de *J.* i *Rowling*, però no és capaç de trobar una relació entre *novel* i *J. K. Rowling*.

ERNIE aprèn amb aquest *marked corpus*:

Harry Potter is a series of [MASK] [MASK] written by [MASK] [MASK] [MASK].

ERNIE permet trobar la relació entre *Harry Potter* i *J. K. Rowling*.

3.3.5. Incidència en la intel·ligència artificial

Els models de llenguatge basats en l'atenció tenen una incidència molt important en la intel·ligència artificial. Un exemple de la millora que representa l'atenció és la cerca d'informació. Al principi, el cercador de Google estava basat en les paraules clau sense context. Amb el cercador actual, un model de llenguatge basat en l'atenció permet entendre el significat «real» de les paraules en context. Així, el cercador entén el que l'usuari cerca, s'anticipa oferint paraules clau que afinen aquesta cerca o bé li mostra directament els resultats més concordes amb les seves intencions.

Un altre exemple de salt qualitatiu és el dels sistemes de pregunta i resposta. Aquest cas es considera un problema de predicció d'una oració, concretament l'oració que, donada una pregunta, és més probable de ser la seva resposta per similitud dels seus focus d'atenció. Amb focus d'atenció oracionals, s'aborden de manera bastant satisfactòria altres temes de la intel·ligència artificial que semblava que havien quedat en dic sec, com establir inferències. Donada una premissa, s'estableix si una hipòtesi és veritable o falsa, o la seva inferència no es pot determinar. La capacitat d'establir inferències és evidentment crucial en el desenvolupament de la intel·ligència artificial i en aplicacions clau, com els diagnòstics a partir d'un volum d'informes escrits per metges. El *notebook* «Atenció i intel·ligència artificial» il·lustra la capacitat dels models d'atenció per a aquestes tasques.

Finalment, cal afegir que també es milloren resultats en tasques de classificació, com l'assignació de polaritat a una opinió en *sentiment analysis*. La capa final és l'encarregada de calcular la probabilitat dels diferents tipus de polaritat per a una opinió. Evidentment, l'elecció de la polaritat més probable serà més ajustada com més ajustada sigui la codificació de l'opinió, això és, com millor s'entengui el seu sentit i la intenció de l'opinador.

3.4. Requisits necessaris

En Peter es queda completament convençut que, amb procediments de *deep learning*, concretament amb *transformers*, les tasques de PLN guanyaran en qualitat. Agraeix les explicacions dels *transformers* i diu:

Bé, ara que ja hem passat per *Barri Sèsam*,* voldria saber si tenim les condicions per implementar aquests procediments a l'empresa. Per exemple, suposem que hem de fer un model de llenguatge per a un client que vol millorar el seu assistent en línia o ens impliquem més en projectes d'intel·ligència artificial relacionats amb inferències, raonament, etc. Estem preparats per fer-ho en un període de temps raonable i amb uns costos assenyats?

* En Peter es refereix als acrònims ELMo i BERT, en un joc dels desenvolupadors amb els noms de personatges de *Barri Sèsam*. Noteu que a la figura 13, a la caixa que correspon al mecanisme BERT, hi apareix la cara d'en Bert, Blas a la versió espanyola de *Barri Sèsam*.

3.4.1. Llicències i corpus font

En Joseph respon que el desenvolupament de models de llenguatge no té cap cost per llicències de programari. Paquets de codi lliure com TensorFlow, Keras o NeMo estan preparats per fer models amb *transformers*.

És més difícil aconseguir els corpus font amb els quals s'entreni el sistema i es creï un model de llenguatge fiable que millori resultats en tasques de PLN i intel·ligència artificial. És cert que Wikipedia ofereix un corpus ingent amb llicència per ser processat sense restriccions. Per al model BERT en anglès, Google va processar la versió anglesa de Wikipedia amb 2,5 milions de paraules més els 800 milions de paraules del corpus de llibres digitalitzats BookCorpus, llibres no publicats i que també són de lliure explotació.*

* <https://bit.ly/3nymx6S>

No obstant això, hi ha dues dificultats. La primera és trobar corpus tan voluminosos en totes les llengües sense restriccions per ser processats. El nombre de fonts disponibles per fer models BERT en castellà, per exemple, no és comparable amb el nombre de fonts disponibles en anglès, i encara més quan es tracta llengües minoritàries com el català o el basc. La Beth apunta que en el cas de les llengües minoritàries és temptador traduir automàticament de l'anglès a la llengua minoritària, amb la qual cosa l'aprenentatge es faria molt probablement amb frases mal traduïdes.

La segona dificultat té a veure amb la preservació de la privadesa i altres assumptes de restriccions legals als quals els textos estan subjectes. No totes les grans institucions i entitats estan disposades a cedir el seu fons documental, i les que ho fan exigeixen que els documents passin per un procés d'anonimització de dades. En aquest cas, l'entrenament es fa amb dades modificades i adulterades d'alguna manera. Aquesta limitació afecta sobretot la confecció de models amb informes mèdics, sentències judicials, etc. No obstant això, hi ha variants BERT per a aquests dominis, com ClinicalBERT (Huang i altres, 2019).

3.4.2. Recursos computacionals

Quant als recursos computacionals, en Joseph li posa l'exemple de la codificació d'una frase de tretze paraules amb el mecanisme d'atenció amb BERT. Cada paraula comporta una *query*, amb la qual cosa la frase tindrà tretze *query*. En cada *query* es computa la rellevància de la paraula multiplicant les matrius Q , K i V . Cal tenir en compte, a més, la multiplicitat de matrius amb pesos $Q/K/V$ calculant l'atenció amb diversos *heads*. D'altra banda, el codificador es compon de N codificadors apilats, cadascun dels quals passa les sortides al codificador següent fins a l'últim, que les passa al descodificador.

BERT pot activar els mecanismes d'atenció en paral·lel, la qual cosa el fa ser més eficient que els LSTM, perquè aquests depenen de la memòria d'una seqüència en un moment donat. Ara bé, la multiplicació de matrius és costosa perquè ha d'obtenir tots els elements de la memòria principal i carregar-los en la unitat de processament.* Google va necessitar quatre dies per entrenar un model BERT de l'anglès bàsic amb 64 GB TPU usant nodes múltiples. Actualment es treballa en l'optimització i la millora en l'arquitectura de BERT, com ha fet Facebook amb la seva RoBERTa.** No obstant això, la generació de models de llenguatge continua essent una tasca que requereix supercomputació per ser rendible; això si és rendible, com diu en Peter, que té la sospita que tots aquests recursos de computació són, segons la tasca, com fer *sentiment analysis*, possiblement com matar mosques a canonades, com se sol dir.

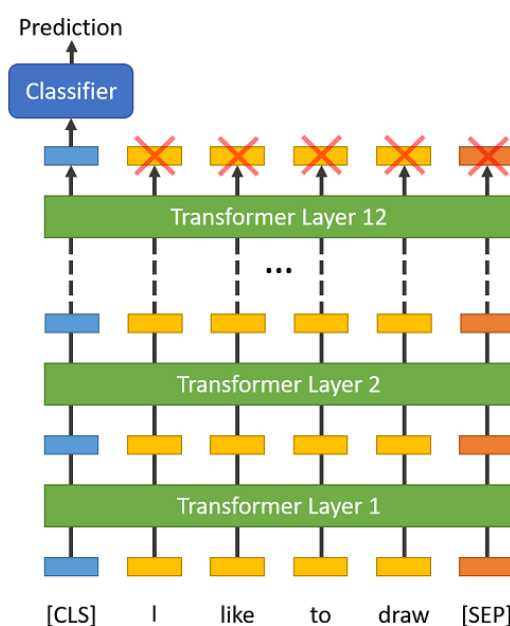
* <https://bit.ly/33EKfGU>
 ** <https://bit.ly/2SFf8EW>

3.5. *Fine-tuning*

En Peter pregunta si per millorar les seves tasques de PLN cal ser Google. La Beth li contesta que una manera d'evitar els costos associats a la creació de models de llengua és aprofitar els models ja fets per companyies amb recursos, com Google i Facebook. Aquests models s'adapten a casos d'ús particulars. Això és l'anomenat *fine-tuning*.

Segons en Joseph, per transferir el model preentrenat de BERT per aplicarlo a una classificació de text, es necessita solament una capa dedicada a la classificació sobre la capa de sortida del *transformer*. A la figura 14 es mostra com el classificador utilitza l'*embedding* corresponent al *token* especial CLS en l'última capa de sortida.

Figura 14. *Workflow output* en BERT i classificador



Font: <https://bit.ly/31letNH>.

L'avantatge d'això és que es poden fer aquestes aplicacions amb models BERT preentrenats en poques hores, sobretot en comparació amb els recursos computacionals i els dies que comporta entrenar un model de llenguatge general des de zero. No obstant això, en Peter, que està acostumat a classificar textos segons la seva polaritat amb recursos com spaCy o aplicant el *bag of words*, es pregunta si val la pena entrenar un model durant dies per arribar a resultats similars. La Beth li replica que el model preentrenat és més precís i està preparat per a qualsevol tasca relacionada amb la intel·ligència artificial.

El *fine-tuning* també és una solució per a les tasques de PLN amb llengües que no són l'anglès. Google ha preparat BERT multilingües* entrenats amb articles de la Wikipedia. Ara bé, de la mateixa manera que hi ha un problema d'infrarepresentació de llengües a la Wikipedia, hi ha pocs documents font disponibles en llengües minoritàries per fer bons models.

* <https://bit.ly/3d7UHKa>

El *fine-tuning* es pot fer amb paquets com PyTorch, que primer creen una instància d'un model, per exemple, un model BERT amb pesos ja precalculats. D'altra banda, els documents als quals s'aplica *fine-tuning* s'han de processar perquè tinguin el format reconeixible pel model carregat. El processament consisteix en passos com la tokenització; afegir els *tokens* CLS i SEP al principi i al final de les oracions, respectivament, i fer els farciments per a les frases amb una longitud inferior al nombre N de dimensions preestablert per als vectors. Després s'associen els *tokens* amb els *tokens* del model per carregar-ne els pesos.

En Peter vol que li aclareixin si el *fine-tuning* no el deslliura d'una infraestructura de supercomputació. Veu models mastodòntics que exigeixen recursos computacionals molt grans i processaments lents. La Beth el tranquil·litza dient que es treballa per fer models BERT més compactes i ràpids, igual d'eficaços però amb menys paràmetres no essencials. Són els anomenats BERT destil·lats,* com DistilBERT.**

* <https://bit.ly/33Hcyob>
** <https://bit.ly/3nwPrUX>

Resum

Aquest mòdul ha presentat el *deep learning* en el PLN des del punt de vista dels aspectes pràctics que comporta adoptar aquest paradigma.

En primer lloc, s'han explicat els avantatges de l'adaptació al *deep learning*. Entre els avantatges destacables hi ha la capacitat de generar models de llenguatge amb els quals es pot entendre una frase segons el context directament del text en brut i sense necessitat d'haver-lo etiquetat prèviament. Encara que dit així pot ser que no ho sembli, la realitat és que el *deep learning* ha significat un avenç espectacular en la resolució de problemes clàssics del PLN, com la resolució anafòrica, la desambiguació, les respostes a preguntes, les inferències entre frases, etc. Això ha fet que el PLN estigui prou madur per donar un gran impuls a la intel·ligència artificial. L'impacte de la traducció automàtica en la millora de la qualitat ha estat evident, però l'impacte en altres tasques de la intel·ligència artificial serà molt potent en els propers anys.

A continuació s'ha parlat dels costos d'adaptació de les dades textuais a dades numèriques i s'ha efectuat un repàs dels diferents models de llenguatge fets amb *deep learning*. Els models de llenguatge fets amb el mecanisme d'*atenció* s'han presentat com els més adequats. Posteriorment, s'han tractat qüestions relatives a l'entrenament d'un model de llenguatge basat en aquest mecanisme.

Finalment, el mòdul s'ha centrat en els recursos necessaris per entrenar un model de llenguatge. Ens hem centrat sobretot en els seus recursos computacionals exigents i hem presentat estratègies com el *fine-tuning* i la destil·lació de models per superar aquestes exigències.

Glossari

fine-tuning *m* Estratègia que consisteix a adaptar els paràmetres precalculats d'un model de llenguatge a unes dades i a una tasca concreta.

model de llenguatge *m* Distribució de probabilitat en una seqüència de paraules.

tokenitzador *m* Mòdul que converteix un text en una seqüència de nombres enters. Cada nombre enter és l'índex d'un *token* en un diccionari.

word embedding *m* Tècnica de processament del llenguatge natural en què les paraules o les frases d'un vocabulari es transformen en vectors amb nombres reals.

Bibliografia

Huang, Kexin; Altosaar, Jaan; Ranganath, Rajesh (2019). *ClinicalBERT: Modeling clinical notes and predicting hospital readmission* [en línia]. <<https://arxiv.org/abs/1904.05342>>

Joshi, Prateek (2019). *How do transformers work in NLP? A guide to the latest state-of-the-art models* [en línia]. <<https://bit.ly/34GuzCp>>

Karkare, Prateek (2019). *Understanding recurrent neural networks in 6 minutes* [en línia]. <<https://bit.ly/33Hq0bK>>

Liang, Xu (2019). *Understand self-attention in BERT intuitively* [en línia]. <<https://bit.ly/36Jz8yg>>

Shukri, Mohd (2019). *How the embedding layers in BERT were implemented* [en línia]. <<https://bit.ly/3ljwEun>>

Vaswani, Ashish; Shazeer, Noam; Parmar, Niki Uszkoreit, Jakob; Jones, Llion; Gomez, Aidan N; Kaiser, Lukasz; Polosukhin, Illia (2017, desembre). «Attention is all you need». *NIPS'17. Proceedings of the 31st International Conference on Neural Information Processing Systems*.

